

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ULAŞIMDA YAPAY ZEKA YARIŞMASI

KRİTİK TASARIM RAPORU

SÜMERZEKA

TAKIM ADI : SümerZEKA

BAŞVURU ID : T3-317064

İÇİNDEKİLER

ŞEKİL LİSTESİ:	2
TABLO LİSTESİ	3
KISALTMALAR:	3
1. TAKIM ŞEMASI:	3
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ	4
3. ALGORİTMALAR VE SİSTEM MİMARİSİ	4
3.1. Veri Setleri:	4
3.1.1. VisDrone:	4
3.1.2. Geçen Sene Yarışmada Kullanılan Resimler:	4
3.1.3. Teknofest Tarafından Verilen Resimler:	4
3.1.4. Kendi Çektiğimiz Resimler:	4
3.2. Algoritmalar	5
3.2.1. Veri Setini Geliştirmek İçin Kullanılan Algoritmalar:	5
3.2.2. Nesne Tespit Algoritmaları:	5
3.2.3. Sistem Mimarisi:	7
4. ÖZGÜNLÜK	8
4.1. NMS optimize edici:	8
4.2. Resim Bozucu ve Blurlayıcılar:	8
4.3. Gece-gündüz resim ayırıcılar:	9
4.4. Birden çok modelin aynı resim üzerinde kullanılması:	9
5. SONUÇLAR VE İNCELEME	10
5.1. İnsan Tespit Modelinin Sonuçları:	10
5.2. Uai ve Uap Tespit Modeli Sonuçları:	11
6. RAPORDA KULLANILAN RESİMLERİN TAM ÇÖZÜNÜRLÜK BAĞLANTI LİNKİ	12
7. KAYNAKÇA	12

ŞEKİL LİSTESİ:

Şekil 1- Takım Görev ve Sorumlulukları	3
Şekil 2- YOLO temsili grip yapısı	6
Şekil 3- Çoklu ve Yanlış İnsan Tespiti	6
Şekil 4- Yarışma Sırası Yapılacak İşler Algoritması	7
Şekil 5- Resim Bozucu ve Bulanıklaştırma Örneği	8
Şekil 6- Bulanıklaştırma Algoritması	9
Şekil 7- Bulanıklaştırma Örneği	9
Şekil 8- İnsan Tespit Sonuçları	10
Şekil 9- Hızlı Hareket Eden Araç Tespiti	10
Şekil 10- Olumsuz Hava Şartlarından Araç Tespiti	11
Şekil 11- Yan yana Duran Çoklu Araç Tespiti	11
Şekil 12- UAP - UAİ Düşük Çözünürlük Tespiti	11
Şekil 13- UAP- UAİ Tespit Sonuçları	12

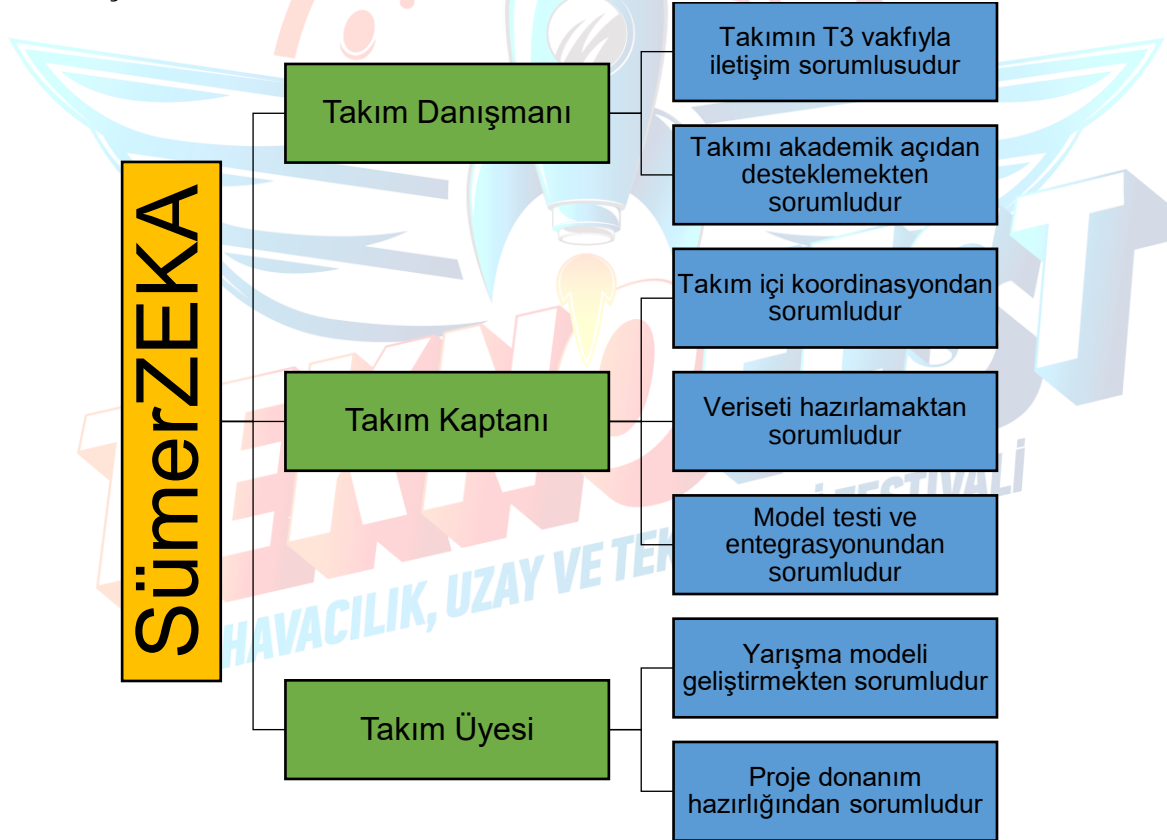
TABLO LİSTESİ:

Tablo 1- SSD Modeli Sonuçları	5
Tablo 2- YoloV5s Modeli Sonuçları	5
Tablo 3- Mimari ve süre karşılaştırması	6

KISALTMALAR:

SSD	: Single Shot Multi Box
YOLO	: You Only Look Once
ÖTR	: Ön Tasarım Raporu
NMS	: Non Maximum Suppression
RCNN	: Region Based Convolutional Neural Networks
CNN	: Convolutional Neural Networks
LR	: Learning Rate
IOU	: Intersection over Union
SIFT	: Scale-Invariant Feature Transform
UAP	: Uçan Araba Park
UAİ	: Uçan Ambulans İniş
FPS	: Frame Per Second

1. TAKIM ŞEMASI:



Şekil 1- Takım Görev ve Sorumlulukları

Şemada gösterildiği üzere takımımız 2 takım üyesi ve 1 takım danışmanından oluşmaktadır. Takımımız yarışmanın başından itibaren takım planına uygun bir şekilde düzenli çalışmalarını sürdürmüştür. Gerekli durumlarda takım üyeleri birlikte çalışmıştır. Takım danışmanımız akademik açıdan literatür taraması, rapor yazımında kaynakların tespiti ve takımın takıldığı noktalarda akademik desteklemekten sorumludur. Takımımız geçtiğimiz yıl da “Ulaşımda Yapay Zeka” yarışmasına katılmıştır.

2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ

Geçtiğimiz yıl model eğitimlerini sahip olduğumuz laptop bilgisayar işlemcisinde eğitmeye çalışmamıza rağmen ana modelimizin eğitimi çok uzun süreceğinden, internet üzerindeki “Sanal makineler” araştırılmıştır. Yaptığımız bu araştırmaların sonucunda Google Colab’da belli süre eğitim yapılmıştır lakin süre kısıtlaması ve yoğunluk nedeniyle yeterince eğitim yapılamamıştır.

Bunun gibi sorunlar göz önünde bulundurularak daha güçlü sistemler satın almak için sponsor aranmıştır. Ön Tasarım Raporumuzun geçmesinin ardından takımımıza Kayseri Melikgazi belediyesi sponsor olarak “RTX 3060” chip setli ekran kartı, anakart, güç kaynağı ve ssd harddisk satın almıştır. Ön Tasarım Raporunda raporumuzun teslimine kadar proje takvimine yetişebilmek için modeller araştırılıp, seçilen modeller eğitilerek bazı testler yapılmıştır. Ön Tasarım Raporunda belirtildiği gibi YOLO algoritması kullanmayı planlıyorduk. Ekran kartının alınması üzerine yarışmadaki sürede daha doğru ve yüksek performanslar elde etmek amacıyla Taşıt, UAİ ve UAP tespiti için YOLO; insan tespiti için Faster R-CNN kullanma kararı alınmıştır. Algoritmalar kısmında bu iki yapının aynı anda nasıl kullanıldığı ayrıntılı bir şekilde açıklanmıştır.

Ön Tasarım Raporunda “Özgünlükler” bölümünde belirttiğimiz sahte resim üreticisinin sorunları tespit edilerek çözülmüştür. Yine ön tasarım raporunda özgünlükler kısmında olan gece gündüz resimleri ayırıcı olarak açık kaynak kodlu hazır bir model kullanmıştık. Bu modeli kullanılmasındaki amaç yarışmada gece resimleri de bulunması durumunda modellerin başarısının artırılması için gece ve gündüz resimlerinden iki farklı model eğitilmesi planlanmaktaydı. Teknik şartnamenin yayınlanmasının ardından yarışmada gece resimlerinin kullanılmayacağı (Yalnızca gün batımı resimlerinin kullanılacağı) kesinleşmiş olduğundan kendi geliştirdiğimiz yapay zeka modeli ile veri setimizdeki gece resimlerini kaldırılmıştır.

Ön Tasarım Raporumuzda yapmayı planladığımız düzenleyici ve ana modelleri destekleyici algoritmaların birçoğu hazırlanıp test edilmiştir. Ayrıca bunun yanı sıra gelişen yeni problemlere uygun çözümler aranıp bulunan çözümler “Özgünlükler” bölümünde ayrıntılı bir şekilde anlatılmıştır. Modellerimizin eğitimi devam etmekte olup, elimizdeki verinin büyütülmesi için internetteki videolardan, resimlerden ve başka dron veri setlerinden alıntılar yapıp takımımız tarafından yarışmamıza uygun olarak etiketlenmektedir.

3. ALGORİTMALAR VE SİSTEM MİMARİSİ

3.1. Veri Setleri:

3.1.1. VisDrone:

Bu veri seti içerisinde barındırdığı fotoğraflar yarışmada istenen formatlara oldukça yakın 8000 fotoğraf bulundurması sebebiyle VisDrone veri setini kullanmaya karar verdik. Fakat VisDrone veri seti yarışmada kullanılmayacak olan gece resimlerini içermektedir. Bizde yazdığımız algoritma sayesinde bu resimleri otomatik ayırarak yarışmada çıkabilecek görüntülere olabildiğince yakın resimleri kullanmaktayız.

3.1.2. Geçen Sene Yarışmada Kullanılan Resimler:

Geçen seneki yarışmada kullanılan resimlerden elimizde kalan yaklaşık 2000 adet resimde etiketleyerek veri setimize dahil ettik.

3.1.3. Teknofest Tarafından Verilen Resimler:

2021 Yılındaki yarışma için Teknofest ekibi tarafından bize verilen örnek videolar elimizde bulunmaktaydı. Bu videoları framelere ayırarak etiketledik ve veri setimizde kullandık.

3.1.4. Kendi Çektiğimiz Resimler:

Okulumuzun TÜBİTAK 4006 sergisinde elimizde bulunan Tello Drone yardımıyla çekimler yaptık. Bu çekimler genel olarak tamamı görünmeyen insanların tespiti için işimize yarayacak. Lakin elimizdeki drone kaliteli çekim yapamadığı için bu resimleri şuan kullanmama kararı aldık. Bunun yerine okulumuzdaki Teknofest 2022 İnsansız Hava Araçları yarışmasına katılacak olan takımımızın yapmış olduğu dronu kullanarak çekimleri tekrarlamayı planlamaktayız.

3.2. Algoritmalar

3.2.1. Veri Setini Geliştirmek İçin Kullanılan Algoritmalar:

3.2.1.1. Resim Çoğaltıcı:

Bu kısımda image augmentation teknikleri kullanılarak resim sayısı özellikle “UAP-UAİ” nesneleri için artırılmıştır. Bu yöntemde resimlerin kontrastı, renkleri ve çözünürlüğü ile oynanarak resim sayısı çoğaltılmaktadır.

3.2.1.2. Etiketlemeyi Kolaylaştırmak İçin Yazdığımız Algoritma:

Etiketleme işleminde hız kazanmak için yazdığımız algoritmayla resimleri geçen seneki modelimiz üzerinden etiketlemekte, ardından yanlış etiketlemeleri elle düzelterek veri setimizi güncellemekteyiz.

3.2.1.3. Veri setleri yapıları arasındaki dönüştürücüler:

Verisetimiz de kendi etiketlemelerimiz yaparken “Labellmg” programı ile YOLO modunda etiketleme yaptık. Tüm veri setimizi YOLO formatında depoladık. Faster-RCNN modeli eğitimi için veri seti kendi yazdığımız kodlar ile dönüştürülmüştür.

Datasetimizin çoğunluğunu oluşturan “Visdrone” veri setinin kendine ait veri yapısı şu şekildedir:(Kordinatlar pixel cinsindendir):

```
<bbox_left>,<bbox_top>,<bbox_width>,<bbox_height>,<score>,<object_category>,<truncation>,<occlusion>
```

YOLO formatı ise (koordinatlar resmin boyutuna oranlıdır):

```
<object_category>,<bbox_center>,<bbox_center>,<bbox_width>,<bbox_height>
```

Kullandığımız diğer bir model olan Faster-RCNN ve diğer Tensorflow modelleri veri yapısı olarak VOC formatındadır.

3.2.2. Nesne Tespit Algoritmaları:

İnsan sınıfı çıkarıldığı zaman geriye kalan “Tasit-Uap-Uai” gibi nesnelerin resimler üzerindeki boyutlarının büyüklüğü ve algılama açısından zorluklarının düşük olması, kendi yaptığımız ve uluslararası çapta birçok veri seti üzerinde yapılan testler sonucu performans ve süre bakımından dengeli olan YOLO ve SSD modellerine öncelik verilmiştir.

Tablo 1- SSD Modeli Sonuçları

Model name	Speed (ms)	COCO mAP	Outputs
SSD ResNet152 V1 FPN 640x640 (RetinaNet152)	80	35.4	Boxes

Tablo 2- YoloV5s Modeli Sonuçları

Model	size (pixels)	mAP ^{val} 0.5:0.95	mAP ^{val} 0.5	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5

Yukarıdaki başarı ve süreler göz önünde bulundurulduğunda YOLOv5 ailesinin giriş modeli YOLOv5s'in Tensorflow 2'in sağladığı en iyi SSD modellerine göre çok daha başarılı olup aynı kategoride bulunduklarından YOLO tercih edilmiştir.

Tablo 3- Mimari ve süre karşılaştırması

Backbone	Top-1	Top-5	Bn Ops	BFLOP/s	FPS
Darknet-19 [15]	74.1	91.8	7.29	1246	171
ResNet-101[5]	77.1	93.7	19.7	1039	53
ResNet-152 [5]	77.6	93.8	29.4	1090	37
Darknet-53	77.2	93.8	18.7	1457	78

Bu başarının en önemli nedenlerinden biri de YOLOV5 in “Backbone” olarak kullandığı Darknet-53 mimarisinin SSD ve Faster RCNN’in kullandığı ResNet-152 mimarisine eşit performansı iki kat hızlı bir şekilde elde edebilmesidir.

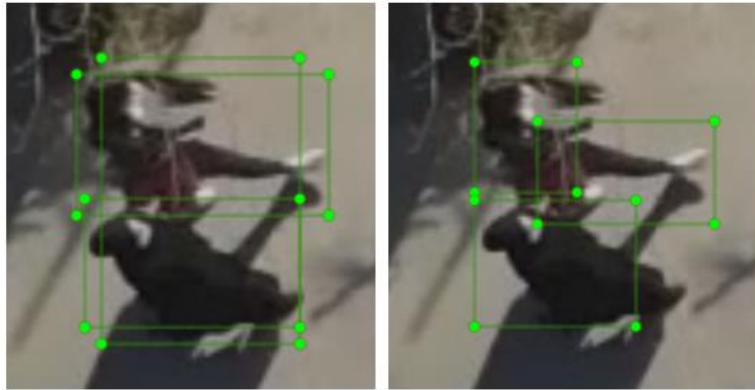


Şekil 2- YOLO temsili grip yapısı

(Temsili olarak YOLO ve SSD bu şekilde gridler kullanmaktadır.)
Yolov5'in grid boyutları sırasıyla: 80*80, 40*40, 20*20 'dir.

Aynı zamanda YOLO ailesinin PyTorch kütüphanesi üzerinde çalışması sebebiyle geliştirilmeye daha açıktır. Bu iki model de ortak olarak farklı boylarda “grid” ler ve “anchor boxe” lar ile tek geçişte nesneler tespit edildiğinden yüksek hızlarda çalışabilmektedir lakin özellikle küçük boyutlu nesnelerin sınırlarının belirlenmesinde ve birbirine yakın nesnelerin ayrılması konusunda sıkıntılar gözlenmiştir.

Bu tip modellerin küçük nesneler üzerinde performanslı çalışmamasının sebeplerinden biri de modelden çıkan tespit kutularının NMS algoritmaları kullanılarak birleştirilerek nesnelerin sınırları belirlenmesidir. Birbirine yakın insanların tespit kutularının birleştirilmesinin engellenmesi için tespit eşiği düşürüldüğü takdirde insan uzuvları birbirinden ayrılabilir.



Şekil 3- Çoklu ve Yanlış İnsan Tespiti

İki sınıf üzerinde ortak eğitilmiş YOLOv5l modeli üzerinde yapılan testler sonucu belirttiğimiz sebeplerden ötürü “İnsan” sınıfı tespitinde başarı oranı düşük olmaktadır. İncelediğimiz diğer modeller arasında insan sınıfının tespitinde en iyi sonuç veren Faster-RCNN modelinin kullanılmasına karar verilmiştir.

RCNN modelleri, günümüzde nesne tanıma alanında çok popüler olan farklı bir yaklaşımdır. Faster-RCNN bu ailenin en yeni üyelerinden olup hız açısından çoğu modelden yavaş olsa da nesne tespit ve sınırlandırma başarısı açısından yaklaşım tekniği tamamıyla farklıdır. Faster RCNN CNN'ler ile çıkardığı Özellik Haritaları üzerinden farklı bir derin öğrenme modeli ile bölge tahmininde bulunmaktadır. Bu bölgeler üzerinde sınıf tahmininde bulunmaktadır. Bölge önerisinin bu denli iyi yapılması sebebiyle insan sınıfı gibi küçük nesnelerde çok başarılı sonuçlar elde edilebilmektedir.

“Uap-Uai” sınıfındaki nesne çeşitliliğinin az olması ve tespit edilmesinin kolay olması sebebiyle taşıt sınıfından ayrılarak daha küçük ve ana modelden yaklaşık olarak %50 daha hızlı bir model olan YOLOv5s kullanılmıştır.

Farklı bir modelle eğitilmesinin önemli diğer bir sebebi ise 1000'lerce resimlik veri setinin çok küçük bir kısmını oluşturduğundan, Kısa süreli eğitimlerde büyük modeller tarafından yeterince kavranılamayıp başarısının düşük olduğu gözlemlendi. Uap-Uai nesnesinin tespit edilmesinin ardından yarışmada ayrıca bu pistlerin inişe uygun olup olmadıklarının da belirlenmesi istenmektedir.

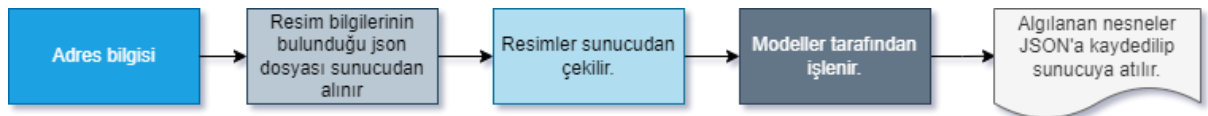
Bu sorunun çözümünde iki farklı yaklaşım denedik:

1. Uap-Uai bölgelerinin tespit edilmesinin ardından, diğer modelimizin bulduğu insan ve taşıtlardan herhangi biri bölge üzerinde bulunmuyorsa bölge “İnişe Uygun” olarak sınıflandırılır.
2. Uap-Uai bulma modelimizi 4 sınıftan oluşturarak inişe uygunluğunda yapay zeka tarafından tespitinin sağlanması. Bu sınıflar: “Bos Uai”, “Bos Uap”, “Dolu Uai”, “Dolu Uap”tır.

Yarışmadaki deneyimlerimiz ve yaptığımız denemeler sonucunda 2. yöntemin daha doğru sonuç verdiği gözlemlenmiştir.

3.2.3. Sistem Mimarisi:

Modellerimiz ve haberleşme kodlarımız yazılırken Python yazılım dili tercih edilmiştir. Kodların küçük bir kısmı C kodları çoğunluğu Python standart kütüphaneleri ve harici bazı kütüphanelerden oluşmaktadır. Eğitim aşamasında Linux tabanlı Debian işletim sistemi kullanılmıştır. Eğitim Cuda sayesinde Ekran Kartı üzerinde yapılmıştır. Bu sayede eğitim hızı artırılmıştır. Eğitim sırasında belli sayıda epoch'tan sonra model hem yerel dosyalara hem de takımımıza ait drive'a yedeklenmiştir. “Acc ve loss” gibi başarı parametreleri grafikler halinde kayıt edilip model başarısı canlı olarak izlenmiştir. Model eğitimi belirli bir seviyeye ulaştığı zaman modelimizin eğitim hyper parametrelerini optimize etmek için Genetik Algoritmalar kullanılmıştır. Eğitim Hyper Parametreleri temelde modelin eğitimi sırasında kullandığı ve eğitilemeyen parametreler (Momentum,lr...) dir. Bu eğitim parametrelerinin optimizasyonun ardından eğitim kaldığı yerden devam ettirilmiştir. Modelimiz daha önce eğitime konulmamış veri setimizden örneklenen resimler üzerinde test edilmiştir. Bu testlerin sonuçları ve süre performans açısından değerlendirmeleri “Sonuçlar ve İnceleme” kısmında gösterilmiştir. Yarışma sırasında ana sunucu ile ethernet protokolü üzerinden haberleşme sağlanacağı teknik şartnamede yer almıştır. Yarışma sırasındaki çalışma planı aşağıdaki tabloda gösterilmiştir.



Şekil 4- Yarışma Sırası Yapılacak İşler Algoritması

4. ÖZGÜNLÜK

4.1. NMS optimize edici:

YOLO modellerinin onbinlerce kutudan oluşan çıktılarını birleştirerek kullanılabilen obje kutularına dönüştüren bu algoritma temelde 3 parametre üzerine kuruludur.

Max_Detect: Maksimum kaç obje kutusu oluşturulacağı.

Conf_Thresh: Modelin bulduğu kutuların minimum sahip olması gereken doğruluk skorudur.

Iou_Thresh: Aynı sınıftan kesişen objelerin "IOU" oranına göre birleştirilme sınırıdır.

Bu parametreler arasından yarışma sonucunu en çok etkileyen Conf ve Iou threshleridir. Ortalama yarışma resimlerindeki obje sayısı <200 olduğu için maksimum obje sayısı 250 civarında seçilmiştir. Teknik şartnamenin incelenmesinin ardından modellerin doğruluğunu kontrol eden bir kod oluşturduk. Yarışma puanlamasına uygun olarak üç adet ödül-ceza parametresi içermektedir

Bunlar:

- KW : Yanlış sınıf tahmini veya "0 iou" için verilen sabit ceza
- KT : Doğru sınıf tahminleri arasından en yüksek iou değerine sahip olan tahminlere verilen ödül
- KM : Doğru sınıfa sahip olmasına rağmen bir nesneye birden çok tahmin gönderilmesi durumunda iou değeri ile ters orantılı verilecek ceza.

Bu 3 parametre ve teknik şartname göz önünde bulundurularak hazırlanan kodumuz seçtiğimiz aralık ve adım büyüklüğüne göre "Iou ve Conf" thresh lerini değiştirerek NMS başarısını optimize etmektedir.

Yaklaşık olarak 500 resim üzerinde yapılan eski modelimizin test sonucu: [Conf:0.25](#) Iou:0.30 olarak hesaplanmıştır. Sonuçlar geçen yıl takımımızın yarışmada kullandığımız değerlere çok yakın bulunmuştur.

Yolo modellerinin eğitiminin ardından NMS optimize edici kullanılarak en uygun ayarlar belirlenecektir.

4.2. Resim Bozucu ve Blurlayıcılar:

Veri setimizdeki fotoğrafların çoğunluğu yüksek kaliteli olmakla birlikte incelediğimiz yarışma resimleri arasında bazı nesnelerin fazla küçük olduğundan veya sarsıntılardan dolayı bulanık çıktığını ve modelimizin bu nesneleri algılamakta çeşitli sıkıntılar çektiğini tespit ettik.

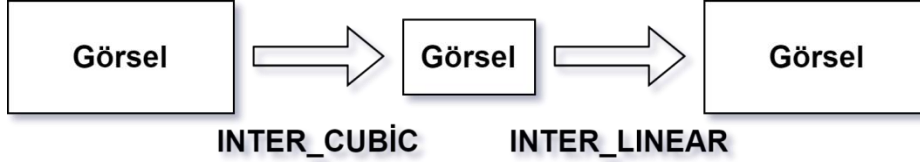


Şekil 5- Resim Bozucu ve Bulanıklaştırma Örneği

Bunun önlenmesi ve modelin nesneleri daha iyi kavrayabilmesi için veri setimizdeki bazı fotoğrafları Median Blur, Gaussian Blur ve Bilateral gibi blur algoritmalarında test ettik. Lakin bu bulanıklaştırma yöntemleri kullanılarak kalitesiz resim etkisini yakalayamadık.

Ardından kendi geliştirdiğimiz yöntem ile resim kalitesi istenildiği gibi düşürülmüştür:

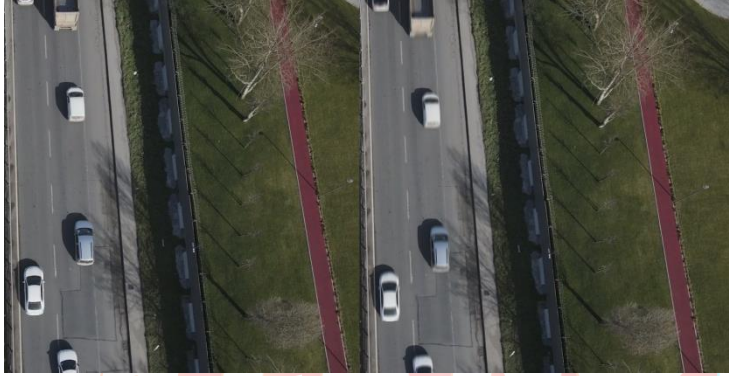
- Öncelikle 8*8 filtre ile ortalama alma yöntemi ile 2 kat küçültülür.
- Ardından resim pixel kopyalama yöntemi ile tekrar eski boyutuna getirilir. Bu sayede resimdeki ayrıntılar azaltılmıştır



Şekil 6- Bulanıklaştırma Algoritması

Yapılan testler sonucu modelimizin hızlı hareket eden araçların motion blur(hareket bulanıklığı) sebebiyle düzgün tespit edilemediği fark edildi. Özellikle düşük fps değerlerinde ve hareketli dronlar tarafından çekilen resimlerin çoğunluğunda bu problem gözlenmiştir. Bu sorunun üstesinden gelebilmek açısından otomatik olarak hareket halinde olan araçlara hareket yönüne zıt ve hızıyla orantılı olarak motion blur ekleyecek bir kod yazılması planlanmıştır.

Kod yazılırken temel problemlerden biri olan dronun da çekim sırasında hareket etmesi olmuştur.



Şekil 7- Bulanıklaştırma Örneği

Yapılan işlemlerin sırası:

1. Resim üzerindeki takibe uygun noktalar SIFT algoritması ile tespit edilmiş ve BFMATCHER algoritması kullanılarak önceki kare ile bir sonraki kare arasındaki noktaların eşleştirilmesi sağlanmıştır
2. Eşleşen her nokta arasındaki x ve y kaymalarının ortalaması alınarak yaklaşık resmin kayması hesaplanmıştır.
3. Ardından taşıt modeli sayesinde araçlar tespit edilip her araç için x ve y

kayması hesaplanıp vektörel çıkarma işlemi yapılmasının ardından kaymanın sınır üzerinde olması durumunda aracın hareket ettiği sonucuna varılmıştır.

4. Seçilen araçların hareket yönünün tersi hesaplanmıştır

5. Opencv kütüphanesi içerisinde bulunan **filter2D** fonksiyonunun içerisine uygun filtreler tanımlanmış ve hesaplanan yöne göre transform edilerek araçlara motion blur etkisi verilmiştir.

4.3. Gece-gündüz resim ayırıcılar:

Resim öncelikli olarak BGR dan HSV ye dönüştürülüp, kendi hazırladığımız conv model ile Parlaklık katmanı alınarak işlenmiş ve resmin gece veya gündüz olduğu tespit edilmiştir. Tespit edilen gündüz resimleri başka bir klasöre aktarılmıştır. Bu sayede 8000 resimden oluşan VisDrone veri seti 6000 resme düşürülmüştür.

4.4. Birden çok modelin aynı resim üzerinde kullanılması:

Algoritmalar kısmında ayrıntılı açıkladığımız sebeplerden dolayı Taşıt-İnsan-(Uap-Uai) nesneleri için her nesnenin kendine has özellikleri göz önünde bulundurularak modeller seçilmiştir. Bir nesne sınıfı için birden çok model kullanılıp bunların birleştirilmesi incelenmiş lakin datasetimizin büyüklüğü ve çok sayıda eğitim gerçekleştirileceği için uygun bulunmamıştır.

5. SONUÇLAR VE İNCELEME

Modellerimizin ekran kartı desteği ile yaklaşık olarak 10000 havadan çekim fotoğraf üzerinde 500 epoch eğitimi tamamlamasının ardından model testlerine başladık. Oluşturduğumuz modellerin yarışma performansının gerçekçi tespit edilebilmesi adına testlerimizi geçmiş yıllarda yarışmada kullanılan fotoğraflar üzerinde gerçekleştirdik. Her test gerçekleştirilirken donanımsal ve yazılımsal aynı şartlar oluşturuldu. Modellerin tüm resimler üzerindeki hatalı bulduğu veya bulamadığı nesneler ayrıntılı incelenip aşağıda detaylı olarak anlatılmıştır. Modellerin fotoğrafları işleme süresi kaydedilip yarışma anında haberleşme süreleri de yaklaşık olarak göz önünde bulundurularak modellerin fps'leri hesaplanmıştır.

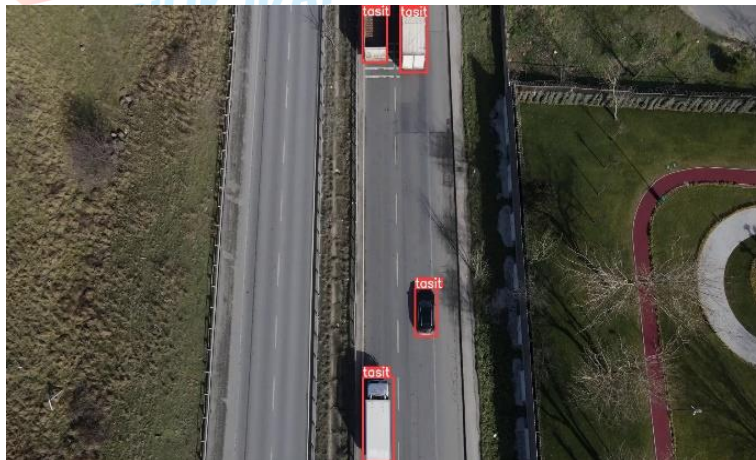
5.1. İnsan Tespit Modelinin Sonuçları:

Modelimiz, birbirine yakın yürümekte olan insanları başarılı bir şekilde sınırlayıp ayırabilmiştir. Ağaç dallarının altında bulunan iki kişi ve ayakta duran bir kişi tespit edebilmektedir. Banklar, çöp kutuları, direkler gibi yanıltıcı nesneler insan olarak tespit edilmemiştir. Elde ettiğimiz bu sonuçlar gösterdi ki taşıt ve insan nesnelerini ayrı modeller halinde eğitmek başarılarımızı büyük oranda arttırdı. Modelimiz "0.4" sn bir frame işlemektedir.



Şekil 8- İnsan Tespit Sonuçları

Modelimizin yüksek hızlardaki taşıtların tespitindeki yeteneğinin ölçülebilmesi için TEKNOFEST'in örnek videosunda bulunan otoyol fotoğrafları üzerinde modelimiz denenmiş olup model çıktı kutuları görsel üzerinde çizdirilmiştir. Taşıtların yüksek yoğunluğu tespit edilip tahmin kutularının kesişim oranları yeterli bulunmuştur. Tespit edilemeyen taşıtlar genellikle fotoğrafların sınırlarında olup az kısmı gözükken araçlardır. Hareketli araçların tespit başarısının daha fazla yükseltilebilmesi için geliştirilen çözümlerimiz özgünlükler bölümünde ayrıntılı anlatılıp veri setinin geliştirilmesinde kullanılmıştır.



Şekil 9- Hızlı Hareket Eden Araç Tespiti



Şekil 10- Olumsuz Hava Şartlarından Araç Tespiti

Yarışma sırasında farklı hava şartlarında çekilmiş fotoğrafların da bulunacağı göz önünde tutularak karlı ve yağmurlu fotoğraflar test edilmiştir. Nesnelerin seçilebilir olduğu durumlarda her görselde taşıtların tespiti yapılabilmektedir. Lakin üzeri karla kaplı araçlardan bazılarının karlı zeminle karıştırılıp tespit edilemediği görülmüştür. Bu sorunun çözümü için karlı havalarda çekilmiş dron videoları fotoğraflarına ayrılıp etiketlenmiş ve veri setine dahil edilmiştir.

Araçların tespitindeki sıkıntılı kısımlardan bir diğeri ise birbirlerine yakın halde bekleyen veya park halinde bulunan araçların birbirlerinden ayrılarak doğru bir biçimde tespit edilmesinin sağlanmasıdır.



Şekil 11- Yan yana Duran Çoklu Araç Tespiti

Bunun için örnek videonun içerisinde bulunan otopark fotoğrafları test edilmiştir. Her aracın sınırı kesin bir şekilde belirlenip tespit edilemeyen araç olmamıştır. Eğitimimizi tamamladıktan sonra özgünlükte detaylı olarak bahsettiğimiz “NMS Optimize Edici” sayesinde tespit parametrelerini optimize ettik. Modelimizin bu resimlerde başarılı olmasındaki önemli sebeplerden biridir.

Modelimiz “0.02 sn” de bir resmi işlemektedir

5.2. Uai ve Uap Tespit Modeli Sonuçları:

Modelimiz bulduğu iniş pistlerini “Uai” veya “Uap” olarak ayırarak bu alanların inişe uygunluğunu başarılı bir şekilde tespit etmektedir. Modelimiz örnek videodaki gerçek boyutlu iniş pistlerini başarılı bir şekilde tespit ettiği gibi bu nesnelere kıyasla oldukça küçük boyuttaki kendi hazırladığımız posterleri de başarılı bir şekilde tespit etmiştir.

Ayrıca modelimiz Şekil 12 de görüldüğü gibi büyük bir çoğunluğu ağaçların altında kalan iniş pistlerini de doğru bir şekilde tespit edebilmiştir.

Modelimiz “0.014 s” de bir resmi işleyebilmektedir.



Şekil 12- UAP - UAI Düşük Çözünürlük Tespiti



Şekil 13- UAP- UAİ Tespit Sonuçları

6. RAPORDA KULLANILAN RESİMLERİN TAM ÇÖZÜNÜRLÜK BAĞLANTI LİNKİ

<https://drive.google.com/drive/folders/1AZOq7z7zi9Df7q2kgtNa-MYIZ052B0gg?usp=sharing>

7. KAYNAKÇA

- <https://colab.research.google.com/> Erişim Tarihi: 10.05.2022
- <https://www.nvidia.com/tr-tr/geforce/graphics-cards/30-series/rtx-3060-3060ti/> Erişim Tarihi: 06.06.2022
- <https://github.com/ultralytics/yolov5> Erişim Tarihi: 12.06.2022
- https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/tf2_detection_zoo.md Erişim Tarihi: 28.04.2022
- <https://pytorch.org/> Erişim Tarihi: 12.05.2022
- <https://github.com/developer0hye/PyTorch-Darknet53> Erişim Tarihi: 12.06.2022
- <https://learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/> Erişim Tarihi:12.06.2022
- https://docs.opencv.org/4.x/da/df5/tutorial_pysift_intro.html Erişim Tarihi:12.06.2022
- https://docs.opencv.org/4.x/d3/da1/classcv_1_1BFMatcher.html Erişim Tarihi:12.06.2022